

Investigación: Problema Complejo Resuelto con POO en Java

1. Contexto del Problema

Problema identificado: Sistema de gestión de transporte público urbano en tiempo real.

Las ciudades latinoamericanas, incluida Costa Rica, enfrentan un reto crítico en la administración de sus redes de autobuses: rutas con decenas de unidades, múltiples paradas, tarifas diferenciadas, miles de pasajeros simultáneos y la necesidad de brindar información actualizada a usuarios y operadores. Hoy en día, gran parte de este sistema se gestiona con hojas de cálculo, llamadas telefónicas y procesos manuales, lo que genera retrasos, ineficiencia operativa y mala experiencia para el pasajero.

El sistema debe manejar de manera coherente:

- Flotas de autobuses con atributos individuales (placa, capacidad, estado, conductor asignado).
- Rutas con múltiples paradas ordenadas y distancias entre ellas.
- Pasajeros con tarjetas de prepago, historial de viajes y saldo.
- Conductores con licencias, horarios y asignaciones.
- Eventos en tiempo real: retrasos, averías, cambios de ruta.

2. Relación con Ingeniería en Computadores

Este problema es inherentemente computacional por varias razones:

- **Concurrencia y estado compartido:** múltiples autobuses actualizan su posición y capacidad simultáneamente; sin sincronización adecuada se producen condiciones de carrera (race conditions).
- **Modelado de entidades del mundo real:** cada autobús, ruta, parada y pasajero es una entidad con estado propio y comportamiento definido, lo que exige abstracciones de software robustas.
- **Escalabilidad:** una ciudad mediana puede tener 500+ unidades y 200 000+ viajes diarios; el diseño debe sostenerse sin degradación.
- **Mantenibilidad:** los requisitos cambian (nuevas tarifas, nuevas rutas); el código debe modificarse con impacto mínimo en el resto del sistema.
- **Integración con hardware y APIs externas:** GPS, validadores de tarjeta, paneles de información, aplicaciones móviles — todos los cuales deben comunicarse con el núcleo del sistema.

La complejidad estructural, el volumen de datos, la necesidad de extensibilidad y la interacción con sistemas externos hacen de este un problema clásico de ingeniería de software donde la Programación Orientada a Objetos (POO) en Java ofrece ventajas claras frente a enfoques procedimentales.

3. Opciones de Modelado con POO en Java

Opción A — Jerarquía de herencia clásica (Herencia + Polimorfismo)

Se define una jerarquía de clases donde los tipos de vehículos, pasajeros y tarifas se especializan mediante herencia. Clases abstractas y métodos polimórficos permiten tratar cualquier unidad de transporte de forma uniforme.

```
import java.util.List;
import java.util.ArrayList;

// Clase base abstracta
abstract class Vehiculo {
    protected String placa;
    protected int capacidadMax;
    protected EstadoVehiculo estado;

    public abstract void actualizar(Evento e);
    public abstract int capacidadDisponible();
}

// Clase derivada
class Autobus extends Vehiculo {
    private List<String> pasajerosBordo; // (Asumí String, cambialo por tu clase Pasajero si tenés una)
    private Ruta rutaAsignada;

    public Autobus() {
        this.pasajerosBordo = new ArrayList<>();
    }

    @Override
    public void actualizar(Evento e) {
        /* lógica específica */
    }

    @Override
    public int capacidadDisponible() {
        return capacidadMax - pasajerosBordo.size();
    }
}
```

Ventajas: reutilización de código, polimorfismo natural en colecciones (List<Vehiculo>), fácil adición de nuevos tipos (p.ej. BusEléctrico extends Autobus).

Desventajas: jerarquías profundas pueden volverse rígidas; Java solo permite herencia simple.

Opción B — Composición + Interfaces (favorece la flexibilidad)

En lugar de heredar comportamiento, cada clase delega responsabilidades a objetos colaboradores. Las interfaces definen contratos (Rastreable, Pagable, Notificable) y las clases los implementan según necesidad, siguiendo el principio 'composición sobre herencia'.

```
import java.util.List;
import java.util.ArrayList;

// Interfaces
interface Rastreable {
    Coordenadas obtenerPosicion();
}

interface Pagable {
    boolean cobrar(double monto);
}

// Implementación de Rastreable
class Autobus implements Rastreable {
    private SensorGPS gps; // composición
    private Ruta ruta;
    private List<String> abordo; // (Asumí String, cambialo por tu objeto Pasajero si es necesario)

    public Autobus() {
        this.abordo = new ArrayList<>();
    }

    @Override
    public Coordenadas obtenerPosicion() {
        return gps.leer();
    }
}

// Implementación de Pagable
class TarjetaPrepago implements Pagable {
    private double saldo;

    @Override
    public boolean cobrar(double monto) {
        if (saldo >= monto) {
            saldo -= monto;
            return true;
        }
        return false;
    }
}
```

Ventajas: bajo acoplamiento, alta cohesión; cada clase tiene una responsabilidad única (SRP); fácil de probar con mocks; extensible sin modificar clases existentes (OCP).

Desventajas: más clases e interfaces; puede ser sobrediseño para sistemas pequeños.

Opción C — Patrón Observer + Strategy para eventos en tiempo real

Los eventos del sistema (avería, retraso, llegada a parada) se modelan con el patrón **Observer**: los autobuses publican

eventos y los suscriptores (paneles de información, apps, centro de control) reaccionan. La lógica de cálculo de tarifa varía según tipo de pasajero (normal, adulto mayor, estudiante) y se encapsula con el patrón **Strategy**.

```
import java.util.List;
import java.util.ArrayList;

// =====
// PATRÓN OBSERVER
// =====

interface Observador {
    void notificar(EventoBus e);
}

class Autobus {
    private List<Observador> suscriptores = new ArrayList<>
();
    public void suscribir(Observador o) {
        suscriptores.add(o);
    }

    public void publicar(EventoBus e) {
        suscriptores.forEach(o -> o.notificar(e));
    }
}

// =====
// PATRÓN STRATEGY
// =====

interface EstrategiaTarifa {
    double calcular(double distanciaKm);
}

class TarifaNormal implements EstrategiaTarifa {
    @Override
    public double calcular(double d) {
        return d * 15.0;
    }
}

class TarifaEstudiante implements EstrategiaTarifa {
    @Override
    public double calcular(double d) {
        return d * 9.0;
    }
}

class Pasajero {
    private EstrategiaTarifa tarifa;

    public Pasajero(EstrategiaTarifa t) {
        this.tarifa = t;
    }

    public double cobrar(double km) {
        return tarifa.calcular(km);
    }
}
```

Ventajas: desacoplamiento total entre emisores y receptores; agregar un nuevo tipo de tarifa o un nuevo suscriptor no requiere modificar clases existentes; ideal para sistemas distribuidos y en tiempo real.

Desventajas: requiere mayor comprensión de patrones de diseño; el flujo de control es menos lineal y puede dificultar la depuración.

Tabla comparativa de las tres opciones

Reutilización	Alta (herencia)	Alta (delegación)	Media
Flexibilidad	Baja-Media	Alta	Muy Alta
Complejidad inicial	Baja	Media	Alta
Adecuada para tiempo real	Parcial	Parcial	Sí
Nivel POO requerido	Básico	Intermedio	Avanzado

La combinación más recomendable para este sistema en Java es **B + C**: composición de objetos con interfaces bien definidas (Opción B) como arquitectura base, y patrones Observer y Strategy (Opción C) para los módulos de eventos y tarifas. La Opción A puede adoptarse puntualmente para la jerarquía de vehículos, donde la herencia es semánticamente correcta. Este enfoque híbrido maximiza la mantenibilidad, la escalabilidad y la testabilidad del sistema, atributos centrales en Ingeniería en Computadores.